



pravinmishraaws / Kubernetes-For-DevOps-Engineers



<> Code

Issues

Pull requests

Actions

Projects

Security

Insights



main



Kubernetes-For-DevOps-Engineers

[/ Section 04: Pods in Kubernetes – Concepts, and Hands-On / Lab](#)[/ Guided Lab 01: Creating Your First Pod in Kubernetes.md](#)

pravinmishraaws Create Guided Lab 01: Creating Your First Pod in Kubernetes.md

a44901c · last month



163 lines (105 loc) · 4.25 KB

Preview

Code

Blame



Raw



Guided Lab 01: Creating Your First Pod in Kubernetes

Understand what a Pod is and how to create one using both imperative and declarative methods.

Welcome to Your First Pod Lab

In Kubernetes, a **Pod** is the smallest, most basic deployable unit. You can think of it like a wrapper around your container — it's what Kubernetes uses to run your app.

Today, we're going to create our very first Pod. And we'll do it in two different ways:

- First, using the **imperative method** — where we tell Kubernetes *exactly* what to do, right from the command line.
- Then, we'll shift to the **declarative approach** — where we write down our desired state in a YAML file and let Kubernetes take it from there.

This lab is not just about running commands. It's about understanding how Kubernetes thinks.

Step 1: Set Up Your Lab Folder

Let's keep things clean and organized. First, open your terminal and create a folder where you'll store all your Pod-related files.

```
mkdir -p ~/k8s-labs/pods  
cd ~/k8s-labs/pods
```



Now you're inside a working directory that's dedicated to Pods. This is where we'll do all our work for this lab.

Part 1: Creating a Pod Imperatively

Let's start with the quickest way to create a Pod — using the `kubectl run` command.

```
kubectl run nginx-pod --image=nginx
```



What's happening here?

- `kubectl run` is a quick way to spin up a Pod.
- `nginx-pod` is the name we're giving to the Pod.
- `--image=nginx` tells Kubernetes to pull the official NGINX image from Docker Hub.

Once you run this, Kubernetes will schedule a Pod named `nginx-pod`, and run a single container based on the NGINX image.

You can check if it's running with:

```
kubectl get pods
```



That's it — you've just created your first Pod.

Now, let's delete it so we can learn how to do the same thing declaratively.

```
kubectl delete pod nginx-pod
```



Part 2: Creating a Pod Declaratively (YAML-based)

The declarative method is how Kubernetes is meant to be used in the real world — especially in production.

Let's write our desired state in a YAML file, then ask Kubernetes to create it for us.

Step 1: Create a new YAML file

```
touch nginx-pod.yaml
```



Now open this file in your preferred editor (`nano` , `vim` , or VS Code). For example:

```
nano nginx-pod.yaml
```



Step 2: Add the following content:

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: nginx
spec:
  containers:
    - name: nginx-container
      image: nginx
      ports:
        - containerPort: 80
```



What's going on in this file?

- `apiVersion: v1` means this is using the core Kubernetes API.
- `kind: Pod` tells Kubernetes we want to create a Pod.
- `metadata.name` sets the Pod's name.
- `labels` help identify this Pod (we'll use this later with Services and Selectors).

- `spec.containers` defines the container(s) inside the Pod. In our case, just one — running the NGINX image.

Step 3: Apply the YAML

Now that we've written our desired state, let's tell Kubernetes to make it real:

```
kubectl apply -f nginx-pod.yaml
```



And check the result:

```
kubectl get pods
```



You should see your `nginx-pod` running just like before — but this time, it came from your YAML definition.

Bonus Step: Peek Into the Pod

Want to see more details about your running Pod?

```
kubectl describe pod nginx-pod
```



Or check logs from the container:

```
kubectl logs nginx-pod
```



Want to get a shell inside the container?

```
kubectl exec -it nginx-pod -- /bin/bash
```



Try browsing to `/usr/share/nginx/html` inside the container. That's where the default NGINX page lives.

Wrap-Up: What Did You Learn?

- A **Pod** is the most basic unit in Kubernetes that runs your container.
- You created a Pod two ways:
 - **Imperative**: Quick, CLI-based
 - **Declarative**: YAML-driven, scalable, and production-ready
- You practiced creating, deleting, inspecting, and debugging a Pod

What's Next?

Next, we'll learn about **ReplicaSets**, which add resilience by ensuring that your Pods stay alive — even if one crashes. We'll also show how this leads naturally to Deployments and rolling updates.